

Databricks

Unified lakehouse platform for data engineering, ML, and analytics on Spark

Overview

Databricks is a managed Apache Spark-based lakehouse platform unifying data engineering, data science, and BI on top of open storage formats (Delta Lake). On Azure it deploys into your subscription as Azure Databricks with native Entra ID and networking integration.

Core Concepts

	Details
Workspace	Control-plane UI for notebooks, jobs, repos — data plane runs in your VNet
Cluster	Set of VMs running Spark; interactive (all-purpose) or job clusters (ephemeral, cheaper)
Delta Lake	ACID transaction layer over Parquet — time travel, schema enforcement, MERGE support
Unity Catalog	Centralized governance: catalog.schema.table namespace, fine-grained access control, lineage
DBFS / Volumes	Distributed file storage abstraction; Volumes provide governed non-tabular file access
Delta Live Tables (DLT)	Declarative pipeline framework with built-in data quality expectations

Medallion Architecture

Bronze (raw ingested data, schema-on-read) → **Silver** (cleaned, validated, conformed schema, deduplicated) → **Gold** (business-level aggregates, ready for BI/ML consumption). Each layer is a Delta table enabling incremental, auditable transformation.

Security & Governance

- **Unity Catalog** — table/column/row-level ACLs, tag-based access, cross-workspace data lineage
- **VNet injection** (Azure) — deploy clusters into your own subnets for NSG/firewall control
- **Secret scopes** — backed by Azure Key Vault; avoid hardcoding credentials in notebooks
- **Cluster policies** — enforce instance types, autotermination, and tag compliance for cost/security control
- **Private Link** — front-end (UI/API) and back-end (data plane to control plane) private connectivity
- **Audit logs** — diagnostic logs stream to Log Analytics/Event Hub for compliance monitoring

PySpark Essentials

Code Pattern	
Read Delta table	<code>spark.read.format("delta").load(path)</code>
Write/merge	<code>DeltaTable.forPath(spark, path).merge(...)</code>
Time travel	<code>spark.read.option("versionAsOf", 3).load(path)</code>
Optimize + Z-order	<code>OPTIMIZE table ZORDER BY (col)</code>
Vacuum	<code>VACUUM table RETAIN 168 HOURS</code> — reclaims stale files
Structured Streaming	<code>readStream.format("delta")...writeStream.trigger(availableNow=True)</code>

CI/CD & MLOps Integration

- **Databricks Asset Bundles** — YAML-based IaC for jobs/pipelines, deployed via CLI in CI/CD
- **MLflow** — native experiment tracking, model registry, and stage transitions (Staging → Production)
- **Repos** — Git-backed notebooks for version control and PR-based review workflows
- **Workflows/Jobs API** — orchestrate multi-task DAGs, integrate with ADF or external schedulers

Common Pitfalls

	Fix
Small files problem degrading query performance	Run OPTIMIZE regularly; use auto-compaction and Z-ordering on filter columns
All-purpose clusters left running idle	Enforce autotermination via cluster policies; prefer job clusters for scheduled work
Notebooks with hardcoded credentials	Use secret scopes backed by Key Vault; never inline connection strings
No lineage/ governance across workspaces	Adopt Unity Catalog early; retrofitting from Hive metastore later is costly
Skipping VACUUM causing storage bloat	Schedule VACUUM with a sensible retention window (min 7 days for time-travel safety)

Interview / Exam Quick Hits

- Delta Lake provides **ACID guarantees** that raw Parquet on a data lake does not
- Job clusters are cheaper than all-purpose clusters because they **terminate automatically** after the job
- Unity Catalog centralizes governance **across workspaces**, replacing legacy per-workspace Hive metastore
- Photon is Databricks' **native vectorized query engine** accelerating SQL and DataFrame workloads
- Auto-scaling clusters balance cost vs. performance but add startup latency — job clusters mitigate with pools